

Library Management System

Project source files: 35 text files

Library-Management-System-main/README.md

Library Management System (Full Stack)

A full-stack borrow/return library system built with the same architecture style as your banking project: Spring Boot REST APIs with a layered design (Controller → Service → Repository), JPA/Hibernate, and a React.js frontend.

Tech Stack

- **Backend:** Java 17, Spring Boot 3, Spring Data JPA / Hibernate, Spring Validation, H2 (in-memory, swappable for MySQL/PostgreSQL), Maven
- **Frontend:** React 18, Axios
- **Architecture:** Layered backend (Controller → Service → Repository), DTOs for request payloads, centralized exception handling (@RestControllerAdvice)

Features

- **Books:** add / list / update / delete, with total vs. available copy tracking
- **Members:** add / list / update / delete
- **Loans:** issue (borrow) a book to a member, return a book, view all loans, view loans by member, and view overdue loans
- Business rules enforced server-side: can't issue a book with 0 available copies, can't return an already-returned loan, duplicate ISBN/email rejected
- Due dates default to 14 days from issue (configurable per loan); loans past their due date are automatically flagged OVERDUE
- CORS enabled so the React app (port 3000) can call the Spring Boot API (port 8080)

Project Structure

```
library-management-system/
├── backend/                                # Spring Boot app
│   ├── pom.xml
│   └── src/main/java/com/libraryapp/
│       ├── entity/                        # Book, Member, Loan (+ enums)
│       ├── repository/                   # Spring Data JPA repositories
│       ├── service/ + service/impl/      # Business logic layer (borrow/return rules)
│       ├── controller/                  # REST controllers
│       ├── dto/                          # Request payloads
│       └── exception/                   # Custom exceptions + global handler
├── frontend/                             # React app
│   ├── src/
│   │   ├── components/                  # BookList, MemberList, LoanList, Navbar
│   │   ├── services/api.js              # Axios wrapper around the REST API
│   │   └── App.js
│   └── README.md
└──
```

Running the Backend

```
bash
cd backend
mvn spring-boot:run
```

- API base URL: `http://localhost:8080/api`
- H2 console: `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:mem:librarydb`, user `sa`, no password)
- Seed data (3 books, 2 members) loads automatically from `data.sql`

Switching to MySQL / PostgreSQL

`application.properties` has a commented block with the MySQL config. Uncomment the `mysql-connector-j` dependency in `pom.xml`, swap in the datasource properties, and point it at a local database.

Running the Frontend

```
bash
cd frontend
npm install
npm start
```

- Opens at `http://localhost:3000`
- `package.json` has `"proxy": "http://localhost:8080"`, so API calls are forwarded to the Spring Boot backend automatically in development.

Key REST Endpoints

Method	Endpoint	Description
POST	<code>/api/books</code>	Add a book
GET	<code>/api/books</code>	List books
PUT	<code>/api/books/{id}</code>	Update a book

```
| DELETE | `/api/books/{id}` | Delete a book |
| POST | `/api/members` | Add a member |
| GET | `/api/members` | List members |
| POST | `/api/loans/issue` | Issue (borrow) a book - `{ "bookId": 1, "memberId": 1, "loanDays": 14 }` |
| POST | `/api/loans/{id}/return` | Return a book |
| GET | `/api/loans` | List all loans (auto-refreshes overdue status) |
| GET | `/api/loans/member/{memberId}` | List a member's loan history |
| GET | `/api/loans/overdue` | List currently overdue loans |
```

Possible Extensions

- Fines/penalties for overdue returns (e.g., a scheduled job + a fine amount on `Loan`)
- Reservations/waitlists when all copies are checked out
- Spring Security + JWT for librarian vs. member roles
- Email/SMS due-date reminders
- Swap H2 for MySQL/PostgreSQL for persistent storage (see above)

Library-Management-System-main/backend/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.2.5</version>
        <relativePath/>
    </parent>

    <groupId>com.libraryapp</groupId>
    <artifactId>library-management-system</artifactId>
    <version>1.0.0</version>
    <name>library-management-system</name>
    <description>Library Management System - Full Stack (Spring Boot + React)</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!-- H2 for easy local run. Swap for MySQL/PostgreSQL in application.properties -->
        <dependency>
            <groupId>com.h2database</groupId>
            <artifactId>h2</artifactId>
            <scope>runtime</scope>
        </dependency>

        <!-- Uncomment to use MySQL instead of H2 -->
        <!--
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        -->

        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
```

```

</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
      </configuration>
    </plugin>
  </plugins>
</build>

</project>

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/LibraryApplication.java

```

package com.libraryapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryApplication.class, args);
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/controller/BookController.java

```

package com.libraryapp.controller;

import com.libraryapp.entity.Book;
import com.libraryapp.service.BookService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/books")
@RequiredArgsConstructor
@CrossOrigin(origins = "")
public class BookController {

    private final BookService bookService;

    @PostMapping
    public ResponseEntity<Book> addBook(@Valid @RequestBody Book book) {
        return new ResponseEntity<>(bookService.addBook(book), HttpStatus.CREATED);
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAllBooks() {
        return ResponseEntity.ok(bookService.getAllBooks());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        return ResponseEntity.ok(bookService.getBookById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> updateBook(@PathVariable Long id, @Valid @RequestBody Book book) {
        return ResponseEntity.ok(bookService.updateBook(id, book));
    }
}

```

```

    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.noContent().build();
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/controller/LoanController.java

```

package com.libraryapp.controller;

import com.libraryapp.dto.IssueLoanRequest;
import com.libraryapp.entity.Loan;
import com.libraryapp.service.LoanService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/loans")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class LoanController {

    private final LoanService loanService;

    @PostMapping("/issue")
    public ResponseEntity<Loan> issueLoan(@Valid @RequestBody IssueLoanRequest request) {
        return new ResponseEntity<>(loanService.issueLoan(request), HttpStatus.CREATED);
    }

    @PostMapping("/{id}/return")
    public ResponseEntity<Loan> returnLoan(@PathVariable Long id) {
        return ResponseEntity.ok(loanService.returnLoan(id));
    }

    @GetMapping
    public ResponseEntity<List<Loan>> getAllLoans() {
        return ResponseEntity.ok(loanService.getAllLoans());
    }

    @GetMapping("/member/{memberId}")
    public ResponseEntity<List<Loan>> getLoansByMember(@PathVariable Long memberId) {
        return ResponseEntity.ok(loanService.getLoansByMember(memberId));
    }

    @GetMapping("/overdue")
    public ResponseEntity<List<Loan>> getOverdueLoans() {
        return ResponseEntity.ok(loanService.getOverdueLoans());
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/controller/MemberController.java

```

package com.libraryapp.controller;

import com.libraryapp.entity.Member;
import com.libraryapp.service.MemberService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/members")
@RequiredArgsConstructor

```

```

@CrossOrigin(origins = "")
public class MemberController {

    private final MemberService memberService;

    @PostMapping
    public ResponseEntity<Member> addMember(@Valid @RequestBody Member member) {
        return new ResponseEntity<>(memberService.addMember(member), HttpStatus.CREATED);
    }

    @GetMapping
    public ResponseEntity<List<Member>> getAllMembers() {
        return ResponseEntity.ok(memberService.getAllMembers());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Member> getMemberById(@PathVariable Long id) {
        return ResponseEntity.ok(memberService.getMemberById(id));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Member> updateMember(@PathVariable Long id, @Valid @RequestBody Member member) {
        return ResponseEntity.ok(memberService.updateMember(id, member));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteMember(@PathVariable Long id) {
        memberService.deleteMember(id);
        return ResponseEntity.noContent().build();
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/dto/IssueLoanRequest.java

```

package com.libraryapp.dto;

import jakarta.validation.constraints.NotNull;
import lombok.Data;

/**
 * Payload used to issue (borrow) a book to a member.
 */
@Data
public class IssueLoanRequest {

    @NotNull(message = "Book id is required")
    private Long bookId;

    @NotNull(message = "Member id is required")
    private Long memberId;

    /** Optional loan period in days; defaults to 14 if not provided. */
    private Integer loanDays;
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/entity/Book.java

```

package com.libraryapp.entity;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

import java.util.List;

@Entity
@Table(name = "books")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Book {

```

```

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@NotBlank(message = "Title is required")
private String title;

@NotBlank(message = "Author is required")
private String author;

@NotBlank(message = "ISBN is required")
@Column(unique = true)
private String isbn;

private String category;

@Min(value = 0, message = "Total copies cannot be negative")
private Integer totalCopies = 1;

@Min(value = 0, message = "Available copies cannot be negative")
private Integer availableCopies = 1;

@OneToMany(mappedBy = "book", cascade = CascadeType.ALL, orphanRemoval = true)
@JsonIgnoreProperties({"book"})
private List<Loan> loans;
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/entity/Loan.java

```

package com.libraryapp.entity;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NoArgsConstructor;

import java.time.LocalDate;

/**
 * Represents a single borrow/return transaction linking a Member to a Book.
 */
@Entity
@Table(name = "loans")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Loan {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "book_id", nullable = false)
    @JsonIgnoreProperties({"loans"})
    private Book book;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "member_id", nullable = false)
    @JsonIgnoreProperties({"loans"})
    private Member member;

    private LocalDate issueDate = LocalDate.now();

    private LocalDate dueDate;

    private LocalDate returnDate;

    @Enumerated(EnumType.STRING)
    private LoanStatus status = LoanStatus.ISSUED;

    public enum LoanStatus {
        ISSUED, RETURNED, OVERDUE
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/entity/Member.java

```
package com.libraryapp.entity;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDate;
import java.util.List;

@Entity
@Table(name = "members")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Member {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Full name is required")
    private String fullName;

    @Email(message = "Valid email is required")
    @Column(unique = true)
    private String email;

    @NotBlank(message = "Phone number is required")
    private String phoneNumber;

    private String address;

    private LocalDate membershipDate = LocalDate.now();

    @OneToMany(mappedBy = "member", cascade = CascadeType.ALL, orphanRemoval = true)
    @JsonIgnoreProperties({"member"})
    private List<Loan> loans;
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/exception/BookUnavailableException.java

```
package com.libraryapp.exception;

public class BookUnavailableException extends RuntimeException {
    public BookUnavailableException(String message) {
        super(message);
    }
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/exception/DuplicateResourceException.java

```
package com.libraryapp.exception;

public class DuplicateResourceException extends RuntimeException {
    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/exception/GlobalExceptionHandler.java

```

package com.libraryapp.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Object> handleNotFound(ResourceNotFoundException ex) {
        return buildResponse(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(DuplicateResourceException.class)
    public ResponseEntity<Object> handleDuplicate(DuplicateResourceException ex) {
        return buildResponse(HttpStatus.CONFLICT, ex.getMessage());
    }

    @ExceptionHandler(BookUnavailableException.class)
    public ResponseEntity<Object> handleUnavailable(BookUnavailableException ex) {
        return buildResponse(HttpStatus.CONFLICT, ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Object> handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new LinkedHashMap<>();
        ex.getBindingResult().getFieldErrors()
            .forEach(err -> errors.put(err.getField(), err.getDefaultMessage()));

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", HttpStatus.BAD_REQUEST.value());
        body.put("errors", errors);
        return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Object> handleGeneric(Exception ex) {
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR, "Something went wrong: " + ex.getMessage());
    }

    private ResponseEntity<Object> buildResponse(HttpStatus status, String message) {
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", LocalDateTime.now());
        body.put("status", status.value());
        body.put("message", message);
        return new ResponseEntity<>(body, status);
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/exception/ResourceNotFoundException.java

```

package com.libraryapp.exception;

public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/repository/BookRepository.java

```

package com.libraryapp.repository;

import com.libraryapp.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

```

```
public interface BookRepository extends JpaRepository<Book, Long> {
    Optional<Book> findByIsbn(String isbn);
    boolean existsByIsbn(String isbn);
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/repository/LoanRepository.java

```
package com.libraryapp.repository;

import com.libraryapp.entity.Loan;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface LoanRepository extends JpaRepository<Loan, Long> {
    List<Loan> findByMemberId(Long memberId);
    List<Loan> findByBookId(Long bookId);
    List<Loan> findByStatus(Loan.LoanStatus status);
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/repository/MemberRepository.java

```
package com.libraryapp.repository;

import com.libraryapp.entity.Member;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface MemberRepository extends JpaRepository<Member, Long> {
    Optional<Member> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/service/BookService.java

```
package com.libraryapp.service;

import com.libraryapp.entity.Book;

import java.util.List;

public interface BookService {
    Book addBook(Book book);
    List<Book> getAllBooks();
    Book getBookById(Long id);
    Book updateBook(Long id, Book book);
    void deleteBook(Long id);
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/service/LoanService.java

```
package com.libraryapp.service;

import com.libraryapp.dto.IssueLoanRequest;
import com.libraryapp.entity.Loan;

import java.util.List;

public interface LoanService {
    Loan issueLoan(IssueLoanRequest request);
    Loan returnLoan(Long loanId);
    List<Loan> getAllLoans();
    List<Loan> getLoansByMember(Long memberId);
    List<Loan> getOverdueLoans();
}
```

```
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/service/MemberService.java

```
package com.libraryapp.service;

import com.libraryapp.entity.Member;

import java.util.List;

public interface MemberService {
    Member addMember(Member member);
    List<Member> getAllMembers();
    Member getMemberById(Long id);
    Member updateMember(Long id, Member member);
    void deleteMember(Long id);
}
```

Library-Management-System-main/backend/src/main/java/com/libraryapp/service/impl/BookServiceImpl.java

```
package com.libraryapp.service.impl;

import com.libraryapp.entity.Book;
import com.libraryapp.exception.DuplicateResourceException;
import com.libraryapp.exception.ResourceNotFoundException;
import com.libraryapp.repository.BookRepository;
import com.libraryapp.service.BookService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class BookServiceImpl implements BookService {

    private final BookRepository bookRepository;

    @Override
    public Book addBook(Book book) {
        if (bookRepository.existsByIsbn(book.getIsbn())) {
            throw new DuplicateResourceException("A book with ISBN " + book.getIsbn() + " already exists");
        }
        if (book.getAvailableCopies() == null) {
            book.setAvailableCopies(book.getTotalCopies());
        }
        return bookRepository.save(book);
    }

    @Override
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @Override
    public Book getBookById(Long id) {
        return bookRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Book not found with id: " + id));
    }

    @Override
    public Book updateBook(Long id, Book updated) {
        Book existing = getBookById(id);

        // Keep availableCopies consistent if totalCopies changes
        int copiesOnLoan = existing.getTotalCopies() - existing.getAvailableCopies();
        existing.setTitle(updated.getTitle());
        existing.setAuthor(updated.getAuthor());
        existing.setIsbn(updated.getIsbn());
        existing.setCategory(updated.getCategory());
        existing.setTotalCopies(updated.getTotalCopies());
        existing.setAvailableCopies(Math.max(0, updated.getTotalCopies() - copiesOnLoan));

        return bookRepository.save(existing);
    }
}
```

```

@Override
public void deleteBook(Long id) {
    Book existing = getBookById(id);
    bookRepository.delete(existing);
}
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/service/impl/LoanServiceImpl.java

```

package com.libraryapp.service.impl;

import com.libraryapp.dto.IssueLoanRequest;
import com.libraryapp.entity.Book;
import com.libraryapp.entity.Loan;
import com.libraryapp.entity.Member;
import com.libraryapp.exception.BookUnavailableException;
import com.libraryapp.exception.ResourceNotFoundException;
import com.libraryapp.repository.BookRepository;
import com.libraryapp.repository.LoanRepository;
import com.libraryapp.repository.MemberRepository;
import com.libraryapp.service.LoanService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.util.List;

/**
 * Handles the borrow / return workflow: checks availability, decrements /
 * increments copies, and tracks due dates & overdue status.
 * Mirrors the layered (Controller -> Service -> Repository) pattern.
 */
@Service
@RequiredArgsConstructor
public class LoanServiceImpl implements LoanService {

    private static final int DEFAULT_LOAN_DAYS = 14;

    private final LoanRepository loanRepository;
    private final BookRepository bookRepository;
    private final MemberRepository memberRepository;

    @Override
    public Loan issueLoan(IssueLoanRequest request) {
        Book book = bookRepository.findById(request.getBookId())
            .orElseThrow(() -> new ResourceNotFoundException("Book not found with id: " +
request.getBookId()));
        Member member = memberRepository.findById(request.getMemberId())
            .orElseThrow(() -> new ResourceNotFoundException("Member not found with id: " +
request.getMemberId()));

        if (book.getAvailableCopies() == null || book.getAvailableCopies() <= 0) {
            throw new BookUnavailableException("No available copies of \"" + book.getTitle() + "\" to issue");
        }

        book.setAvailableCopies(book.getAvailableCopies() - 1);
        bookRepository.save(book);

        int loanDays = request.getLoanDays() != null ? request.getLoanDays() : DEFAULT_LOAN_DAYS;

        Loan loan = new Loan();
        loan.setBook(book);
        loan.setMember(member);
        loan.setIssueDate(LocalDate.now());
        loan.setDueDate(LocalDate.now().plusDays(loanDays));
        loan.setStatus(Loan.LoanStatus.ISSUED);

        return loanRepository.save(loan);
    }

    @Override
    public Loan returnLoan(Long loanId) {
        Loan loan = loanRepository.findById(loanId)
            .orElseThrow(() -> new ResourceNotFoundException("Loan not found with id: " + loanId));

        if (loan.getStatus() == Loan.LoanStatus.RETURNED) {
            throw new BookUnavailableException("This loan has already been returned");
        }

        loan.setReturnDate(LocalDate.now());
    }
}

```

```

        loan.setStatus(Loan.LoanStatus.RETURNED);
        loanRepository.save(loan);

        Book book = loan.getBook();
        book.setAvailableCopies(book.getAvailableCopies() + 1);
        bookRepository.save(book);

        return loan;
    }

    @Override
    public List<Loan> getAllLoans() {
        refreshOverdueStatuses();
        return loanRepository.findAll();
    }

    @Override
    public List<Loan> getLoansByMember(Long memberId) {
        return loanRepository.findByMemberId(memberId);
    }

    @Override
    public List<Loan> getOverdueLoans() {
        refreshOverdueStatuses();
        return loanRepository.findByStatus(Loan.LoanStatus.OVERDUE);
    }

    /** Marks any ISSUED loan whose due date has passed as OVERDUE. */
    private void refreshOverdueStatuses() {
        List<Loan> issued = loanRepository.findByStatus(Loan.LoanStatus.ISSUED);
        LocalDate today = LocalDate.now();
        for (Loan loan : issued) {
            if (loan.getDueDate() != null && loan.getDueDate().isBefore(today)) {
                loan.setStatus(Loan.LoanStatus.OVERDUE);
                loanRepository.save(loan);
            }
        }
    }
}

```

Library-Management-System-main/backend/src/main/java/com/libraryapp/service/impl/MemberServiceImpl.java

```

package com.libraryapp.service.impl;

import com.libraryapp.entity.Member;
import com.libraryapp.exception.DuplicateResourceException;
import com.libraryapp.exception.ResourceNotFoundException;
import com.libraryapp.repository.MemberRepository;
import com.libraryapp.service.MemberService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
@RequiredArgsConstructor
public class MemberServiceImpl implements MemberService {

    private final MemberRepository memberRepository;

    @Override
    public Member addMember(Member member) {
        if (memberRepository.existsByEmail(member.getEmail())) {
            throw new DuplicateResourceException("A member with email " + member.getEmail() + " already exists");
        }
        return memberRepository.save(member);
    }

    @Override
    public List<Member> getAllMembers() {
        return memberRepository.findAll();
    }

    @Override
    public Member getMemberById(Long id) {
        return memberRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Member not found with id: " + id));
    }
}

```

```

@Override
public Member updateMember(Long id, Member updated) {
    Member existing = getMemberById(id);
    existing.setFullName(updated.getFullName());
    existing.setEmail(updated.getEmail());
    existing.setPhoneNumber(updated.getPhoneNumber());
    existing.setAddress(updated.getAddress());
    return memberRepository.save(existing);
}

@Override
public void deleteMember(Long id) {
    Member existing = getMemberById(id);
    memberRepository.delete(existing);
}
}

```

Library-Management-System-main/backend/src/main/resources/data.sql

```

INSERT INTO books (title, author, isbn, category, total_copies, available_copies) VALUES
('Clean Code', 'Robert C. Martin', '9780132350884', 'Software Engineering', 3, 3),
('Effective Java', 'Joshua Bloch', '9780134685991', 'Java', 2, 2),
('Spring in Action', 'Craig Walls', '9781617297571', 'Java', 2, 2);

INSERT INTO members (full_name, email, phone_number, address, membership_date) VALUES
('Rahul Sharma', 'rahul.sharma@example.com', '9876543210', 'Andheri, Mumbai', CURRENT_DATE),
('Priya Verma', 'priya.verma@example.com', '9123456780', 'Kothrud, Pune', CURRENT_DATE);

```

Library-Management-System-main/frontend/package.json

```

{
  "name": "library-management-frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "axios": "^1.6.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "eslintConfig": {
    "extends": ["react-app"]
  },
  "browserslist": {
    "production": [">0.2%", "not dead", "not op_mini all"],
    "development": ["last 1 chrome version", "last 1 firefox version", "last 1 safari version"]
  },
  "proxy": "http://localhost:8080"
}

```

Library-Management-System-main/frontend/public/index.html

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>Library Management System</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
  </body>
</html>

```

Library-Management-System-main/frontend/src/App.css

```
.container {
  max-width: 1050px;
  margin: 24px auto;
  padding: 0 20px 60px;
}

.page h2 {
  margin-bottom: 16px;
  color: #3d2817;
}

.card {
  background: #fff;
  border-radius: 10px;
  padding: 20px;
  box-shadow: 0 1px 4px rgba(0,0,0,0.08);
  margin-bottom: 24px;
}

.form-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 12px;
  align-items: center;
}

input, select {
  padding: 10px 12px;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  font-size: 0.9rem;
}

.btn-primary {
  background-color: #3d2817;
  color: #fff;
  border: none;
  padding: 10px 16px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: 600;
}

.btn-primary:hover { background-color: #2b1c10; }

.btn-secondary {
  background-color: #e5e7eb;
  color: #1f2937;
  border: none;
  padding: 10px 16px;
  border-radius: 6px;
  cursor: pointer;
}

.table {
  width: 100%;
  border-collapse: collapse;
  background: #fff;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 1px 4px rgba(0,0,0,0.08);
}

.table th, .table td {
  text-align: left;
  padding: 12px 14px;
  border-bottom: 1px solid #eef0f3;
  font-size: 0.9rem;
}

.table th {
  background-color: #f1f5f9;
  color: #374151;
  font-weight: 600;
}

.empty {
  text-align: center;
  color: #9ca3af;
  padding: 24px;
}

.link {
  background: none;
  border: none;
  color: #3d2817;
}
```

```

    cursor: pointer;
    margin-right: 10px;
    font-size: 0.85rem;
    padding: 0;
}
.link:hover { text-decoration: underline; }
.link.danger { color: #dc2626; }
.link.disabled { color: #9ca3af; cursor: not-allowed; text-decoration: none; }

.badge {
    padding: 3px 10px;
    border-radius: 999px;
    font-size: 0.75rem;
    font-weight: 600;
}
.badge-active { background-color: #dcfce7; color: #166534; }
.badge-closed { background-color: #fee2e2; color: #991b1b; }
.badge-pending { background-color: #fef3c7; color: #92400e; }

.error {
    background-color: #fee2e2;
    color: #991b1b;
    padding: 10px 14px;
    border-radius: 6px;
    margin-bottom: 16px;
    font-size: 0.9rem;
}

```

Library-Management-System-main/frontend/src/App.js

```

import React, { useState } from 'react';
import Navbar from './components/Navbar';
import BookList from './components/BookList';
import MemberList from './components/MemberList';
import LoanList from './components/LoanList';
import './App.css';

export default function App() {
    const [active, setActive] = useState('books');

    return (
        <div>
            <Navbar active={active} onNavigate={setActive} />
            <main className="container">
                {active === 'books' && <BookList />}
                {active === 'members' && <MemberList />}
                {active === 'loans' && <LoanList />}
            </main>
        </div>
    );
}

```

Library-Management-System-main/frontend/src/components/BookList.js

```

import React, { useEffect, useState } from 'react';
import { getBooks, createBook, updateBook, deleteBook } from '../services/api';

const emptyForm = { title: '', author: '', isbn: '', category: '', totalCopies: 1 };

export default function BookList() {
    const [books, setBooks] = useState([]);
    const [form, setForm] = useState(emptyForm);
    const [editingId, setEditingId] = useState(null);
    const [error, setError] = useState('');

    const loadBooks = async () => {
        try {
            const res = await getBooks();
            setBooks(res.data);
        } catch (err) {
            setError('Could not load books. Is the backend running on :8080?');
        }
    };

    useEffect(() => { loadBooks(); }, []);

    const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

    const handleSubmit = async (e) => {

```

```

e.preventDefault();
setError('');
try {
  const payload = { ...form, totalCopies: Number(form.totalCopies) };
  if (editingId) {
    await updateBook(editingId, payload);
  } else {
    await createBook(payload);
  }
  setForm(emptyForm);
  setEditingId(null);
  loadBooks();
} catch (err) {
  setError(err.response?.data?.message || 'Failed to save book');
}
};

const handleEdit = (book) => {
  setForm({
    title: book.title,
    author: book.author,
    isbn: book.isbn,
    category: book.category || '',
    totalCopies: book.totalCopies,
  });
  setEditingId(book.id);
};

const handleDelete = async (id) => {
  if (!window.confirm('Delete this book?')) return;
  await deleteBook(id);
  loadBooks();
};

return (
  <div className="page">
    <h2>Books</h2>
    {error && <div className="error">{error}</div>}

    <form onSubmit={handleSubmit} className="card form-grid">
      <input name="title" placeholder="Title" value={form.title} onChange={handleChange} required />
      <input name="author" placeholder="Author" value={form.author} onChange={handleChange} required />
      <input name="isbn" placeholder="ISBN" value={form.isbn} onChange={handleChange} required />
      <input name="category" placeholder="Category" value={form.category} onChange={handleChange} />
      <input name="totalCopies" type="number" min="1" placeholder="Total copies" value={form.totalCopies}
onChange={handleChange} required />
      <button type="submit" className="btn-primary">
        {editingId ? 'Update Book' : 'Add Book'}
      </button>
      {editingId && (
        <button type="button" className="btn-secondary" onClick={() => { setForm(emptyForm);
setEditingId(null); }}>
          Cancel
        </button>
      )}
    </form>

    <table className="table">
      <thead>
        <tr>
          <th>Title</th><th>Author</th><th>ISBN</th><th>Category</th><th>Available / Total</th><th></th>
        </tr>
      </thead>
      <tbody>
        {books.map((b) => (
          <tr key={b.id}>
            <td>{b.title}</td>
            <td>{b.author}</td>
            <td>{b.isbn}</td>
            <td>{b.category}</td>
            <td>
              <span className={`badge ${b.availableCopies > 0 ? 'badge-active' : 'badge-closed'}`}>
                {b.availableCopies} / {b.totalCopies}
              </span>
            </td>
            <td>
              <button className="link" onClick={() => handleEdit(b)}>Edit</button>
              <button className="link danger" onClick={() => handleDelete(b.id)}>Delete</button>
            </td>
          </tr>
        ))}
        {books.length === 0 && (
          <tr><td colspan="6" className="empty">No books yet.</td></tr>
        )}
      </tbody>
    </table>
  </div>

```

```

    </table>
  </div>
);
}

```

Library-Management-System-main/frontend/src/components/LoanList.js

```

import React, { useEffect, useState } from 'react';
import { getLoans, issueLoan, returnLoan, getBooks, getMembers } from '../services/api';

const emptyForm = { bookId: '', memberId: '', loanDays: 14 };

export default function LoanList() {
  const [loans, setLoans] = useState([]);
  const [books, setBooks] = useState([]);
  const [members, setMembers] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [error, setError] = useState('');

  const loadAll = async () => {
    try {
      const [loanRes, bookRes, memberRes] = await Promise.all([
        getLoans(), getBooks(), getMembers(),
      ]);
      setLoans(loanRes.data);
      setBooks(bookRes.data);
      setMembers(memberRes.data);
    } catch (err) {
      setError('Could not load data. Is the backend running on :8080?');
    }
  };

  useEffect(() => { loadAll(); }, []);

  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      await issueLoan(form);
      setForm(emptyForm);
      loadAll();
    } catch (err) {
      setError(err.response?.data?.message || 'Failed to issue loan');
    }
  };

  const handleReturn = async (id) => {
    try {
      await returnLoan(id);
      loadAll();
    } catch (err) {
      setError(err.response?.data?.message || 'Failed to return book');
    }
  };

  const availableBooks = books.filter((b) => b.availableCopies > 0);

  const isOverdue = (loan) =>
    loan.status === 'ISSUED' && new Date(loan.dueDate) < new Date();

  return (
    <div className="page">
      <h2>Loans (Borrow / Return)</h2>
      {error && <div className="error">{error}</div>}

      <form onSubmit={handleSubmit} className="card form-grid">
        <select name="bookId" value={form.bookId} onChange={handleChange} required>
          <option value="">Select book</option>
          {availableBooks.map((b) => (
            <option key={b.id} value={b.id}>{b.title} ({b.availableCopies} available)</option>
          ))}
        </select>
        <select name="memberId" value={form.memberId} onChange={handleChange} required>
          <option value="">Select member</option>
          {members.map((m) => <option key={m.id} value={m.id}>{m.fullName}</option>)}
        </select>
        <input name="loanDays" type="number" min="1" placeholder="Loan days" value={form.loanDays}
        onChange={handleChange} />
        <button type="submit" className="btn-primary">Issue Book</button>
      </form>
    </div>
  );
}

```

```

<table className="table">
  <thead>
    <tr>
      <th>Book</th><th>Member</th><th>Issue Date</th><th>Due Date</th>
      <th>Return Date</th><th>Status</th><th></th>
    </tr>
  </thead>
  <tbody>
    {loans.map((l) => (
      <tr key={l.id}>
        <td>{l.book?.title}</td>
        <td>{l.member?.fullName}</td>
        <td>{l.issueDate}</td>
        <td>{l.dueDate}</td>
        <td>{l.returnDate || '-'}</td>
        <td>
          <span className={`badge ${
            l.status === 'RETURNED' ? 'badge-active'
            : isOverdue(l) || l.status === 'OVERDUE' ? 'badge-closed'
            : 'badge-pending'
          }`}>
            {l.status === 'ISSUED' && isOverdue(l) ? 'OVERDUE' : l.status}
          </span>
        </td>
        <td>
          {l.status !== 'RETURNED' && (
            <button className="link" onClick={() => handleReturn(l.id)}>Return</button>
          )}
        </td>
      </tr>
    ))}
    {loans.length === 0 && (
      <tr><td colSpan="7" className="empty">No loans yet.</td></tr>
    )}
  </tbody>
</table>
</div>
);
}

```

Library-Management-System-main/frontend/src/components/MemberList.js

```

import React, { useEffect, useState } from 'react';
import { getMembers, createMember, updateMember, deleteMember } from '../services/api';

const emptyForm = { fullName: '', email: '', phoneNumber: '', address: '' };

export default function MemberList() {
  const [members, setMembers] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [editingId, setEditingId] = useState(null);
  const [error, setError] = useState('');

  const loadMembers = async () => {
    try {
      const res = await getMembers();
      setMembers(res.data);
    } catch (err) {
      setError('Could not load members. Is the backend running on :8080?');
    }
  };

  useEffect(() => { loadMembers(); }, []);

  const handleChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      if (editingId) {
        await updateMember(editingId, form);
      } else {
        await createMember(form);
      }
      setForm(emptyForm);
      setEditingId(null);
      loadMembers();
    } catch (err) {
      setError(err.response?.data?.message || 'Failed to save member');
    }
  };
}

```

```

const handleEdit = (member) => {
  setForm({
    fullName: member.fullName,
    email: member.email,
    phoneNumber: member.phoneNumber,
    address: member.address || '',
  });
  setEditingId(member.id);
};

const handleDelete = async (id) => {
  if (!window.confirm('Delete this member?')) return;
  await deleteMember(id);
  loadMembers();
};

return (
  <div className="page">
    <h2>Members</h2>
    {error && <div className="error">{error}</div>}

    <form onSubmit={handleSubmit} className="card form-grid">
      <input name="fullName" placeholder="Full name" value={form.fullName} onChange={handleChange} required
    />
      <input name="email" type="email" placeholder="Email" value={form.email} onChange={handleChange}
    required />
      <input name="phoneNumber" placeholder="Phone number" value={form.phoneNumber} onChange={handleChange}
    required />
      <input name="address" placeholder="Address" value={form.address} onChange={handleChange} />
      <button type="submit" className="btn-primary">
        {editingId ? 'Update Member' : 'Add Member'}
      </button>
      {editingId && (
        <button type="button" className="btn-secondary" onClick={() => { setForm(emptyForm);
setEditingId(null); }}>
          Cancel
        </button>
      )}
    </form>

    <table className="table">
      <thead>
        <tr><th>Name</th><th>Email</th><th>Phone</th><th></th></tr>
      </thead>
      <tbody>
        {members.map((m) => (
          <tr key={m.id}>
            <td>{m.fullName}</td>
            <td>{m.email}</td>
            <td>{m.phoneNumber}</td>
            <td>
              <button className="link" onClick={() => handleEdit(m)}>Edit</button>
              <button className="link danger" onClick={() => handleDelete(m.id)}>Delete</button>
            </td>
          </tr>
        ))}
        {members.length === 0 && (
          <tr><td colspan="4" className="empty">No members yet.</td></tr>
        )}
      </tbody>
    </table>
  </div>
);
}

```

Library-Management-System-main/frontend/src/components/Navbar.js

```

import React from 'react';

export default function Navbar({ active, onNavigate }) {
  const tabs = [
    { key: 'books', label: 'Books' },
    { key: 'members', label: 'Members' },
    { key: 'loans', label: 'Loans' },
  ];

  return (
    <nav style={styles.nav}>
      <div style={styles.brand}>■ Library Management System</div>
      <div style={styles.tabs}>
        {tabs.map((tab) => (
          <button

```

```

        key={tab.key}
        onClick={() => onNavigate(tab.key)}
        style={{
          ...styles.tabButton,
          ...(active === tab.key ? styles.tabActive : {}),
        }}
      >
        {tab.label}
      </button>
    ))}
  </div>
</nav>
);
}

const styles = {
  nav: {
    display: 'flex',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: '14px 28px',
    backgroundColor: '#3d2817',
    color: '#fff',
    flexWrap: 'wrap',
  },
  brand: { fontWeight: 700, fontSize: '1.1rem' },
  tabs: { display: 'flex', gap: '8px' },
  tabButton: {
    background: 'transparent',
    border: '1px solid rgba(255,255,255,0.4)',
    color: '#fff',
    padding: '8px 16px',
    borderRadius: '6px',
    cursor: 'pointer',
    fontSize: '0.9rem',
  },
  tabActive: {
    backgroundColor: '#fff',
    color: '#3d2817',
    fontWeight: 600,
  },
};

```

Library-Management-System-main/frontend/src/index.css

```

* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
  background-color: #f4f6f9;
  color: #1f2937;
}

```

Library-Management-System-main/frontend/src/index.js

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

Library-Management-System-main/frontend/src/services/api.js

```

import axios from 'axios';

// Talks to the Spring Boot backend (Controller -> Service -> Repository layers).
// CRA's "proxy" field in package.json forwards /api calls to :8080 in development.
const api = axios.create({
  baseURL: '/api',
  headers: { 'Content-Type': 'application/json' },
});

```

```
});
```

```
// ---- Books ----
```

```
export const getBooks = () => api.get('/books');  
export const createBook = (data) => api.post('/books', data);  
export const updateBook = (id, data) => api.put(`/books/${id}`, data);  
export const deleteBook = (id) => api.delete(`/books/${id}`);
```

```
// ---- Members ----
```

```
export const getMembers = () => api.get('/members');  
export const createMember = (data) => api.post('/members', data);  
export const updateMember = (id, data) => api.put(`/members/${id}`, data);  
export const deleteMember = (id) => api.delete(`/members/${id}`);
```

```
// ---- Loans ----
```

```
export const getLoans = () => api.get('/loans');  
export const issueLoan = (data) => api.post('/loans/issue', data);  
export const returnLoan = (id) => api.post(`/loans/${id}/return`);  
export const getOverdueLoans = () => api.get('/loans/overdue');
```

```
export default api;
```